Python Engineering - Week 01

Week 01 - Logic and fundamentals

```
01_python_bootcamp/week_01_logic/
```

How to use this document

This PDF is a review index for the week. Each class summarizes **the problem, concepts and syntax/functions introduced**. If a concept becomes fuzzy, locate it here and return to the original exercise.

class_01_equation_to_algorithm

EX01 - Dynamic pressure

Problem: Convert an engineering equation into a sequence of Python instructions.

Concepts: problem -> inputs -> equation -> algorithm -> output; variables; assignment; numerical expressions.

Types: int = integer; float = decimal/real number; str = text; bool = True/False.

Syntax: x = value, operators + - * / ** and print(...).

Key idea: A variable is a name referring to a value; a mathematical equation must become executable operations.

class_02_conditionals

EX02 Rocket equation | EX03 Quadratic solver | EX04 Drag coefficient vs Mach

Problem: Make the program choose different paths depending on data values.

Concepts: boolean conditions, comparisons, exclusive branches, special cases and physical/mathematical domains.

Syntax: if, elif, else; ==, !=, <, <=, >, >=; and / or / not.

Applications: Tsiolkovsky, quadratic discriminant and piecewise drag coefficient versus Mach.

Key idea: Program flow can change according to a condition.

class_03_loops

EX05 Sum of squares | EX06 Factorial | EX07 Taylor exp | EX08 Vertical motion + Mini Project 01

Problem: Repeat an algorithm and accumulate/evolve results.

Concepts: iteration, counter, accumulator, iteration count, convergence and state update.

Syntax: for, range(...), accumulator variables and updates inside the loop.

Numerical methods: Taylor series for exp(x) and Euler method for vertical motion.

Project: Ballistic Flight Simulator v1 with dt sensitivity/convergence study.

Key idea: A loop repeats a block; in numerical methods each iteration often represents one model step.

class_04_functions

EX09-EX12 - Engineering functions and vertical motion

Problem: Encapsulate calculations for reuse and reduced duplication.

Concepts: function, parameters, arguments, return, local scope, validation and multiple outputs.

Syntax: def function(...):, return value and function calls.

Patterns: validate_inputs(), guard clauses and multiple values returned as a tuple.

Applications: dynamic pressure, rocket equation, Taylor exp and vertical motion reorganized into functions.

Key idea: A function should have one clear responsibility, defined inputs and a useful output.

Quick concept index

Concept	Where to find it
Problem: Convert an engineering equation into a sequence of Python instructions.	class_01_equation_to_algorithm
Concepts: problem -> inputs -> equation -> algorithm -> output; variables; assignment; numerical expressions.	class_01_equation_to_algorithm
Types: int = integer; float = decimal/real number; str = text; bool = True/False.	class_01_equation_to_algorithm
Syntax: x = value, operators + - * / ** and print(...).	class_01_equation_to_algorithm
Key idea: A variable is a name referring to a value; a mathematical equation must become executable operations.	class_01_equation_to_algorithm
Problem: Make the program choose different paths depending on data values.	class_02_conditionals
Concepts: boolean conditions, comparisons, exclusive branches, special cases and physical/mathematical domains.	class_02_conditionals
Syntax: if, elif, else; ==, !=, <, <=, >, >=; and / or / not.	class_02_conditionals
Applications: Tsiolkovsky, quadratic discriminant and piecewise drag coefficient versus Mach.	class_02_conditionals
Key idea: Program flow can change according to a condition.	class_02_conditionals
Problem: Repeat an algorithm and accumulate/evolve results.	class_03_loops
Concepts: iteration, counter, accumulator, iteration count, convergence and state update.	class_03_loops
Syntax: for, range(...), accumulator variables and updates inside the loop.	class_03_loops
Numerical methods: Taylor series for exp(x) and Euler method for vertical motion.	class_03_loops
Project: Ballistic Flight Simulator v1 with dt sensitivity/convergence study.	class_03_loops
Key idea: A loop repeats a block; in numerical methods each iteration often represents one model step.	class_03_loops
Problem: Encapsulate calculations for reuse and reduced duplication.	class_04_functions
Concepts: function, parameters, arguments, return, local scope, validation and multiple outputs.	class_04_functions
Syntax: def function(...):, return value and function calls.	class_04_functions

Review rule

First locate **the concept**; then open that class PDF; finally run or modify the exercise. Document + code is the review mechanism for this course.